

Toward Automating Cooja Experiment Workflows for Dataset Generation

Greta Grigoryan*, Hrant Khachatryan*[†], Theofanis P. Raptis[‡]

*Yerevan State University, Yerevan, Armenia. Email: {grigoryan.greta, hrant.khachatryan}@ysu.am

[†]YerevaNN, Yerevan, Armenia

[‡]Institute of Informatics and Telematics, National Research Council, Pisa, Italy. Email: theofanis.raptis@iit.cnr.it

Abstract—The development of Internet of Things (IoT) applications requires rigorous testing and validation, often facilitated by simulation environments. These environments abstract real-world systems, enabling the synthesis and automation of complex experiments without the need for direct interaction with physical hardware. Cooja, an extensible and versatile IoT simulator within the Contiki operating system, supports the emulation of diverse network topologies and protocols, making it an essential tool for IoT research. However, Cooja lacks an end-to-end automated experiment generation and management mechanism, necessitating manual configuration and management that can be both labor-intensive and error-prone. This paper presents an initial design and implementation of an automated system for experiment generation and management within Cooja. Our approach aims to streamline experimental workflows, enhance efficiency, and reduce the overhead associated with manual processes. This work represents a significant step toward fully automating IoT experiment workflows in Cooja, fostering more efficient and reliable IoT research and development.

Index Terms—Internet of Things, Wireless Sensor Networks, Simulation

I. INTRODUCTION

Simulation environments play a crucial role in the research and development of Internet of Things (IoT) applications [1]. These environments offer an abstracted layer where various IoT use cases can be rigorously tested and validated without the need for direct interaction with physical hardware. This abstraction is not only cost-effective but also time-efficient, as it allows for the synthesis and automation of complex experiments that would be impractical or impossible to conduct in the real world [2]. By leveraging simulation environments, researchers and developers can iterate rapidly, identify potential issues, and refine their solutions in a controlled and repeatable manner.

One prominent simulation tool that facilitates this process is Cooja, an extensible and versatile simulator specifically designed for IoT networks, part of the Contiki operating system [3]. Cooja provides a flexible platform for simulating various types of IoT devices and their interactions within a network. It supports the emulation of hardware and network layers, enabling detailed and accurate representations of real-world scenarios. By utilizing Cooja, researchers can model complex IoT environments, conduct experiments with different network

topologies and protocols, and analyze the performance and behavior of their systems under diverse conditions [4].

Synthetic dataset generation within such simulation environments can significantly enhance the development of machine learning (ML) models [5]. By creating comprehensive and varied datasets that represent a wide range of potential real-world conditions, researchers can train ML models to be more robust and generalizable [6]. This approach ensures that models are better prepared to handle the unpredictability and variability of real-world data, thereby improving their efficacy in solving practical problems. Synthetic datasets not only accelerate the training process but also help in overcoming the limitations of scarce or hard-to-obtain real-world data, paving the way for more effective and innovative IoT solutions.

Despite the advantages offered by Cooja, there is a notable gap in its capabilities: the lack of an end-to-end automated experiment generation and management mechanism [7]. This gap typically forces users to manually configure and manage their experiments, which is both labor-intensive and prone to human error [8]. In this paper, we address this critical issue by presenting the initial design and implementation of an automated system for experiment generation and management within the Cooja environment. Our proposed solution aims to streamline the experimental workflow, reduce the overhead associated with manual configuration, and enhance the overall efficiency and reliability of IoT research conducted using Cooja.

We note that there have already been efforts for experiment automation in Cooja, but the use case verticals were quite dedicated and the scope of those researches was quite different. For example, in [9], the authors present a multi-level tracing approach that enable data logging at different levels while maintaining a global time, targeting mostly intrusion detection use cases. In [10], the same authors follow-up by incorporating also ML models to act on the data. And in [11], the authors focus on the generation of new labelled security-specific datasets. In [12], the authors focus on agricultural use cases and extend Cooja to automatically include additional types of data, such as weather and soil data. Contrary to those works, we try to expand our scope to a more horizontal, use case-independent approach, specifically tailored for generalized ML applications, moving the focus from dataset generation to end-

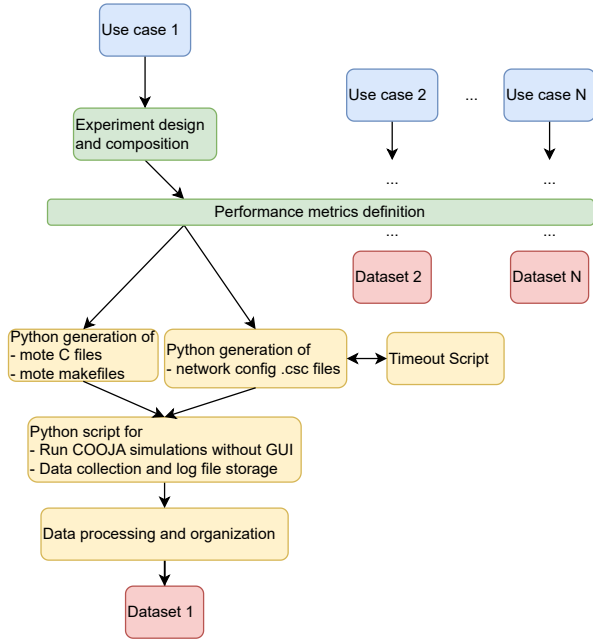


Fig. 1: Experiment workflow automation.

to-end larger-scale experiment automation.

In section II, we present the high level workflow design of our approach. In section III, we dive into the technical description of our implementation. In section IV, we present the currently implemented spectrum of parameter handling for automated experiments. In section V, we describe the plotting and visualization aspects. Lastly, in VI, we conclude the paper.

II. HIGH-LEVEL WORKFLOW DESCRIPTION

This section presents the high-level workflow of our approach to automating experiments in Cooja¹. Fig. 1 illustrates the workflow we have been using for designing, executing, and analyzing experiments with Cooja and Python scripts. The process ensures a structured approach to dataset generation and processing from different use cases.

The *Use Case* blocks include descriptions and functional requirements that guide the design, execution, and processing of experiments to produce the needed datasets. The workflow consists of several key component blocks, each contributing to the overall process.

At the *Experiment Design and Composition* block, we set up our experiment by establishing the necessary parameters and code structure. In the *Performance Metrics Definition*, we identify key performance indicators and carry out the experiment. The *Python Generation of Files* block is responsible for:

- *Mote C Files and Makefiles*: A Python script is used to generate .c files for motes, as well as their corresponding makefiles. These files are crucial for defining the behavior of individual motes in the simulation.
- *Network Configuration .csc Files*: Another Python script is responsible for generating network configuration .csc files. These files define the network topology and other parameters necessary for the simulation. The previously generated .c files are integrated into these configuration files.

The *Timeout Script* block, from the “Network Configuration .csc Files” branch, adds a timeout script to the .csc files. This script ensures that the simulations have a predefined duration, which is essential for consistency and comparability of results. The *Running Simulations and Data Collection* block is responsible for:

- *Run Cooja Simulations without GUI*: Both the “Mote C Files and Makefiles” and “Network Configuration .csc Files” branches converge into a single branch where a Python script is used to run the Cooja simulations without the graphical user interface (GUI). This script automates the execution of simulations, making the process more efficient.
- *Data Collection and Log File Storage*: During the simulation runs, data is collected and stored in log files. This step is critical for capturing the output of the experiments for subsequent analysis.

Once the log files are collected, the *Data Processing and Organization* block processes the data. This step involves separating and organizing the data according to predefined criteria, ensuring that only relevant information is kept for further analysis. The final block compiles the processed data into *Dataset* blocks. These datasets represent the outcome of the entire workflow and contain all the necessary information based on the “Use Case” requirements.

The described steps from “Use Case” to “Dataset 1” can be repeated for different use cases (Use Case 1, ..., Use Case N), each generating a corresponding dataset (Dataset 1, ..., Dataset N). This iterative process allows for the comparison and evaluation of different scenarios, ultimately contributing to a comprehensive analysis of the experiments.

III. TECHNICAL DESCRIPTION OF THE IMPLEMENTATION

In this section, we present the technical aspects of our implementation. In order to demonstrate the added value of our automation, we present both methods for running simulations in Cooja: with the use of the GUI (III-A) and our automated implementation without GUI (III-B). The original GUI-based approach is ideal for beginners, providing a visual interface for setting up and running simple simulations. On the other hand, our non-GUI approach is more suitable for automated and large-scale experiments where a visual interface is not necessary.

¹Our implementation can be found at <https://github.com/Gre-ta/Cooja-auto>.

A. Simulation with GUI

To start the simulation software, open a terminal window and enter the following commands: `cd contiki-ng/tools/cooja` `./gradlew run`

This will launch Cooja with an empty window, as shown in Fig. 2. To create a new simulation, click on **File** → **New Simulation**. Choose a name for the simulation, set any optional parameters, and click **Create** (see Fig. 3). This action will bring up the initial simulation screen, depicted in Fig. 4.

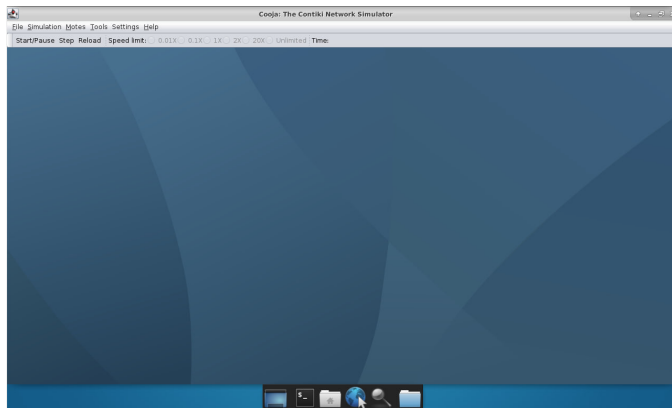


Fig. 2: Initial Cooja screen.

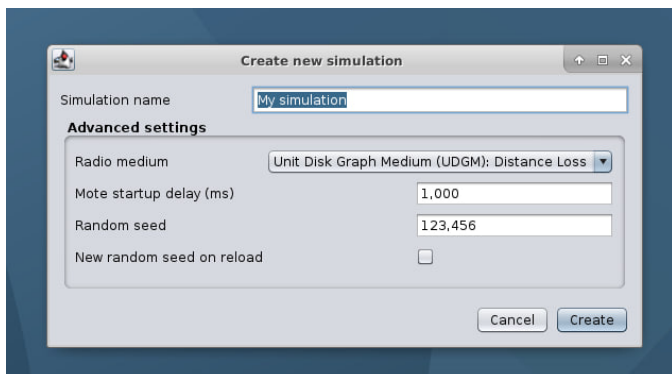


Fig. 3: Cooja new simulation creation.

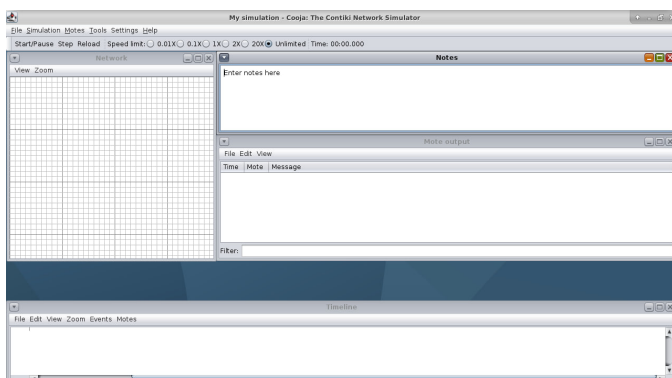


Fig. 4: Initial Cooja simulation screen.

At this point, there is nothing to run since no motes have been added to the network yet. To add motes, click on **Motes**, then **Add motes**, followed by **Create new mote type**, and select a mote type (for example, Sky mote) from the dropdown menu, as illustrated in Fig. 5. In the window that opens, click **Browse** and add the **nullnet-broadcast.c** script for motes from the Contiki-NG examples file (see Fig. 7). After this, click **Compile** and wait until the compilation is complete, then click **Create** (Fig. 6). Next, specify the number of

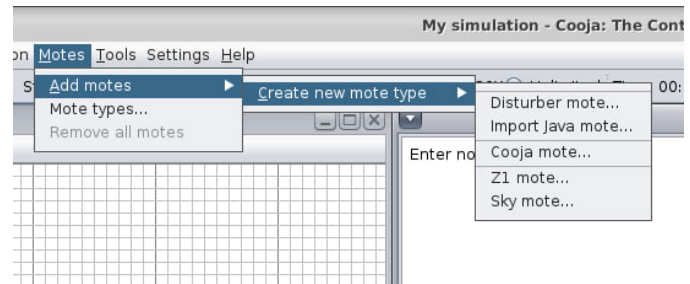


Fig. 5: Cooja add mote dropdown menu.

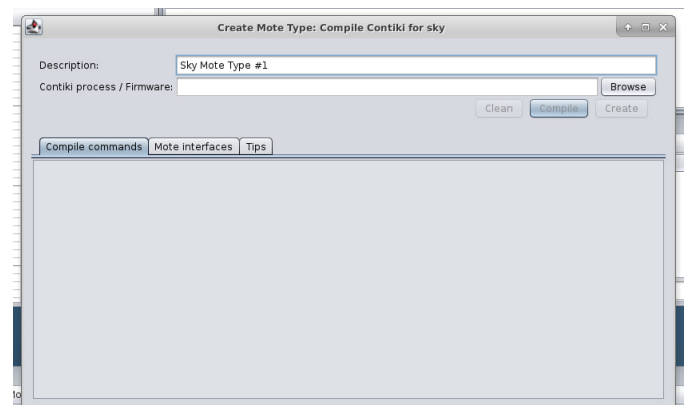


Fig. 6: Cooja locate mote firmware.

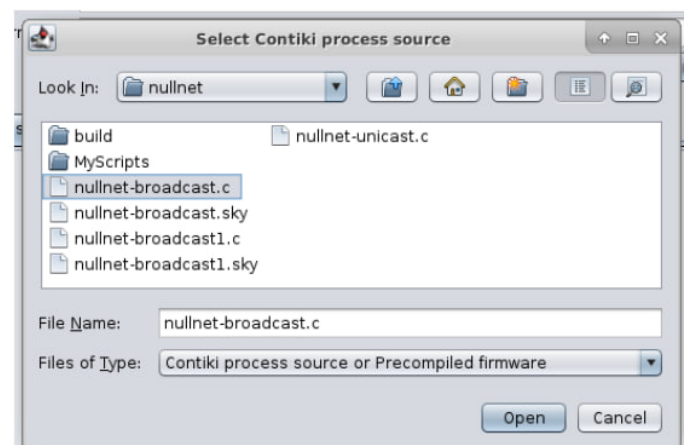


Fig. 7: Nullnet directory.

motes required, select their arrangement, and click **Add motes** (Fig. 8). Once the network is created, click **Start/Pause** to initiate the simulation and run it for as long as desired. To stop the simulation, click the same button again. After stopping, save the simulation by clicking **File** → **Save As**, and save it as a .csc file (e.g., in the directory `/home/vagrant/contiki-ng/tools/cooja/`). Finally, close the Cooja Simulator.

Screenshots of different network configurations created using the Cooja Simulator are shown in Fig. 9 and Fig. 10). Each network consists of several nodes with the nullnet-broadcast.c script added. The experiments were run for approximately 30 minutes, and the results were saved as .csc files. These images illustrate the topologies and node placements used in our experiments.

B. Simulation without GUI

After creating the .csc file, we can run the simulation without using the GUI. But before doing that, we need to add a small script to adjust the time at the end of the .csc files (SimulationAutomationScriptForCSC.py). After that step, we need to open the Terminal and type the following commands: `cd contiki-ng/tools/cooja` and `./gradlew run -args='-no-gui /home/vagrant/contiki-ng/tools/cooja/Example1.csc'`.

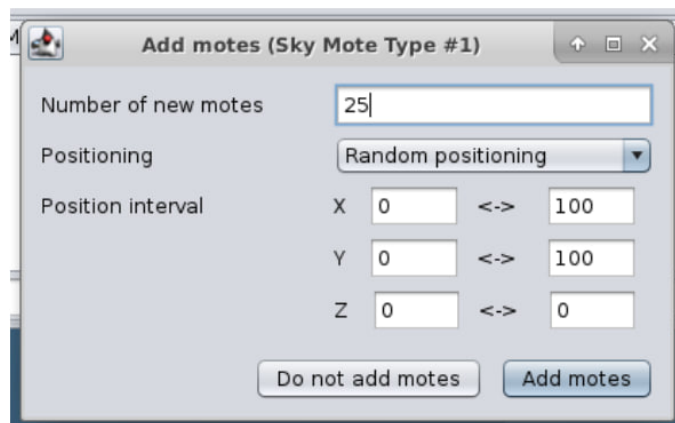


Fig. 8: Mote adding tab.

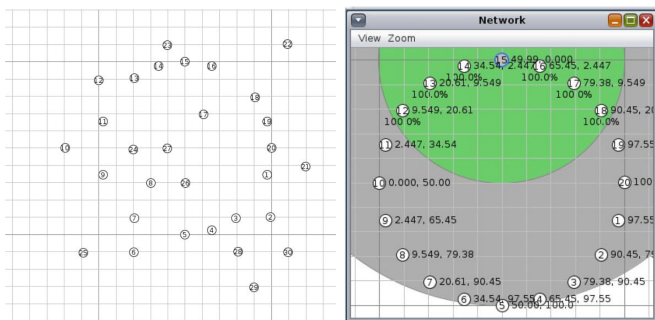


Fig. 9: Network Example 1.

When it's done, we can open the Logfile by typing the following command: `less COOJA.testlog`.

At this point, we can use a simple Python script to run the simulation without the GUI, open the LogFile, and then save the LogFile at the desired directory. The Python script example that we used for this purpose is the SimulationAutomation.py.

IV. PARAMETER SETTING

With the prepared .csc and .c files, we want to make various parameters change systematically so that comprehensive data analysis could be enabled. We now detail the adjustment method for each parameter.

A. Energest data log

Energest is a third-party tool for Cooja designed to monitor and report the energy consumption of various components in a mote². It provides detailed statistics on CPU usage, LPM, Deep LPM, and radioactivity. By integrating Energest into our experiments, we can gain insights into the energy efficiency and performance of the network. To record CPU usage, Low Power Mode (LPM), and Deep LPM data in the log file, it is necessary to integrate a specific script into the .c file used in the simulation. This script will facilitate the logging of these metrics, allowing for a detailed analysis of power consumption and CPU activity. By including this script, the simulation will output the relevant data to the log file, providing insights into:

- CPU Usage: The time the CPU spends in active mode.
- Low Power Mode (LPM): Period of time the mote is in low-power mode.
- Deep LPM: Instances when the mote enters a deeper low-power state.

²Energest can be found here: <https://docs.contiki-ng.org/en/develop/doc/programming/Energest.html>.

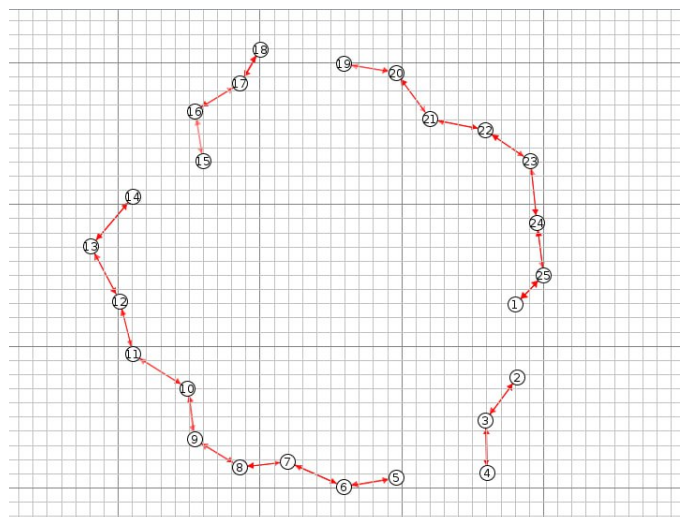


Fig. 10: Network Example 2.

- **Additional Metrics:** Other parameters concerning the power states and activity of the CPU.

This modification ensures that the log file contains comprehensive data on the system's power states, which is essential for analyzing performance and energy efficiency (EnergestForC). This script outputs data at periodic intervals, including: (i) The time (in seconds) that the CPU has been active, in sleep mode, and in deep sleep mode, (ii) The time (in seconds) that the radio has been off, listening, or transmitting. This detailed logging allows for an in-depth analysis of CPU and radio activity over time, providing insights into the system's performance and power consumption (Fig. 11).

B. Message Sending Frequency

The frequency of message sending can be adjusted directly in the .c file. To do this, we locate the following line in the code: `#define SEND_INTERVAL (8*CLOCK_SECOND)` and replace the 8 with the desired interval (in seconds) to modify the message sending frequency according to our use case requirements. This modification will adjust how often messages are sent during a simulation. The clock library defines a macro, `CLOCK_SECOND`, to convert seconds into the tick resolution of the platform. Typically this is 1-10 milliseconds, e.g. `4*CLOCK_SECOND` could be 512. A 16 bit counter would thus overflow every 1-10 minutes. Platforms use the tick interrupt to maintain a long term count of seconds since startup.

C. Transmitting and interference Ranges

To modify the transmitting and interference ranges, we access the .csc files used in the simulation and locate the following lines:

```
<transmitting_range>50.00</transmitting_range>
<interference_range>50.00</interference_range>
```

These lines specify a transmitting and interference range of 50.00 meters. To adjust these ranges, simply change the numerical values to the desired distances.

D. Duration of simulation

To change the duration of the simulation, we modify the script added to the .csc files used in the automation process. We locate or include the following line in the script:

```
443 10317430 20 Energest:
444 10323376 20 CPU          0s LPM          9s DEEP LPM    0s Total time 10s
445 10323635 42 Energest:
446 10327951 20 Radio LISTEN 10s TRANSMIT 0s OFF          0s
447 10329581 42 CPU          0s LPM          9s DEEP LPM    0s Total time 10s
448 10334156 42 Radio LISTEN 10s TRANSMIT 0s OFF          0s
449 10357734 10 Energest:
450 10363680 10 CPU          0s LPM          9s DEEP LPM    0s Total time 10s
451 10368255 10 Radio LISTEN 10s TRANSMIT 0s OFF          0s
452 10432592 22 Energest:
453 10438538 22 CPU          0s LPM          9s DEEP LPM    0s Total time 10s
454 10443113 22 Radio LISTEN 10s TRANSMIT 0s OFF          0s
455 10449133 35 Energest:
456 10455079 35 CPU          0s LPM          9s DEEP LPM    0s Total time 10s
457 10459654 35 Radio LISTEN 10s TRANSMIT 0s OFF          0s
458 10467576 40 Energest:
459 10473522 40 CPU          0s LPM          9s DEEP LPM    0s Total time 10s
460 10478097 40 Radio LISTEN 10s TRANSMIT 0s OFF          0s
461 10491981 24 Energest:
462 10497927 24 CPU          0s LPM          9s DEEP LPM    0s Total time 10s
```

Fig. 11: Log output with Energest.

`TIMEOUT(2000000)`

In the default settings, 1 tick typically equals 1 millisecond. Therefore, this line represents 2,000,000 milliseconds, or 2,000 seconds (which is equivalent to 33 minutes and 20 seconds). Adjust this value as needed to set the desired simulation duration.

E. Number of motes

1) *Fixed Number:* To automatically generate .csc files with a fixed number of modes, where positions are determined automatically, you can use a Python script. This script performs the following tasks:

- 1) *Define the Number of Modes:* Specify the desired number of modes, e.g., `num_motes = 30`.
- 2) *Generate Random Positions:* Assign random positions to each node within the simulation area.
- 3) *Create Simulation XML Content:* Construct the XML content for the simulation, including the node positions, node configurations, and any necessary plugins such as `ScriptRunner`.
- 4) *Save the XML Content:* Save the generated XML content as a .csc file in a specified directory.

This approach allows for efficient and automated setup of simulation scenarios with the required number of modes and their configurations (`FixedNumberOfModes.py`).

2) *Random Number:* To enhance the diversity of our simulations, we implemented a Python script that automatically generates .csc files with a random number of motes. This approach allows for a wide range of scenarios to be tested, each with varying network sizes and configurations. The script dynamically determines the number of motes for each simulation, ensuring that each .csc file is unique. By using this method, we can analyze network performance under different conditions. This automated generation of .csc files not only saves time but also ensures a degree of randomness and variability in our experimental setup (`RandomNumberOfMotes.py`).

F. Network Topology

Network topology defines the arrangement and connectivity of the nodes within the network, directly influencing communication efficiency, data transmission reliability, and overall network performance. Different network topologies, such as line, grid, tree, and ring, each have unique characteristics that make them suitable for specific use cases.

1) *Line Topology:* One of the topologies used in our simulations is the line topology. We generated .csc files with this topology automatically using the `LineTopology.py` script, which creates (x, y) coordinates for the specified number of motes, arranges them in a straight line, and calculates their distances accordingly.

2) *Grid Topology:* Another topology employed in our simulations is the grid topology. For this, we used a Python script to automatically generate .csc files. We used the script

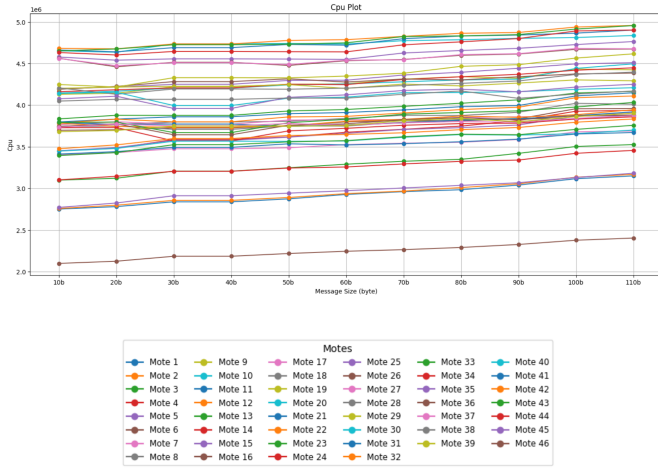


Fig. 12: Impact of Message Size on CPU Time Usage.

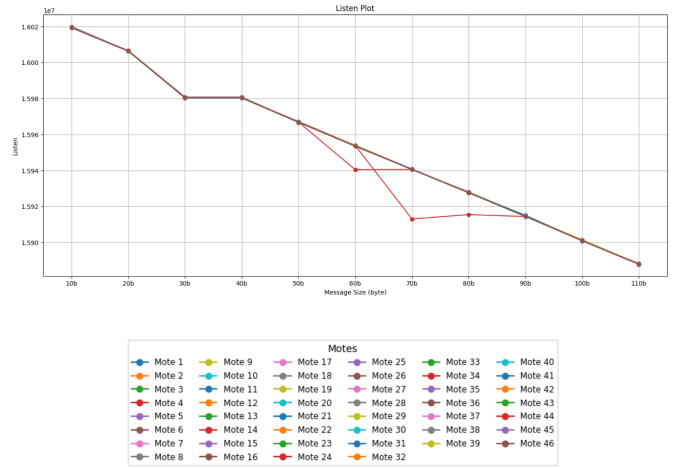


Fig. 14: Impact of Message Size on Listen Time Usage.

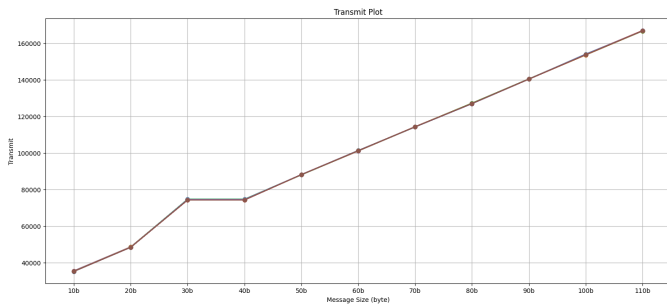


Fig. 13: Impact of Message Size on Transmit Time Usage.

GridTopology.py which arranges the motes in a grid pattern by calculating (x, y) coordinates based on a specified grid size.

G. Size of Message

In this section, we discuss another important parameter we modified: the message size. To adjust this, we modified our .c file by including the following lines:

- **#define MESSAGE_SIZE 110**: This defines the size of the message buffer.
- **static uint8_t message[MESSAGE_SIZE]**: This declares the message array with a size of MESSAGE_SIZE.
- **nullnet_len = MESSAGE_SIZE**: This sets the length of the nullnet_buf to MESSAGE_SIZE.

By increasing the message size from 10 to 110 bytes, we observed a significant impact on CPU time (see Fig. 12). Specifically, the transmit time increased (see Fig. 13), while the listen time decreased (see Fig. 14). These changes reflect how message size influences network performance and resource consumption.

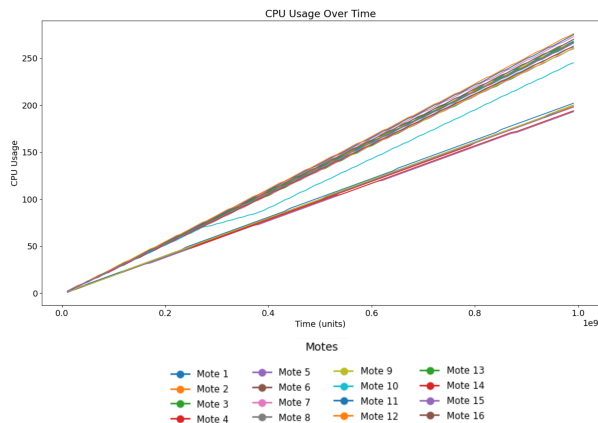
	time	mote	cpu	lpm	deephpm	total
1	10350181	24	0	9	0	10
2	10387923	8	0	9	0	10
3	10505614	11	0	9	0	10
4	10551274	2	0	9	0	10
5	10565810	6	0	9	0	10
6	10641659	18	0	9	0	10
7	10663512	4	0	9	0	10
8	10697350	1	0	9	0	10
9	10716924	7	0	9	0	10
10	10765251	14	0	9	0	10
11	10777703	15	0	9	0	10
12	10782647	25	0	9	0	10
13	10787634	20	0	9	0	10
14	10788096	16	0	9	0	10
15	10871634	10	0	9	0	10
16	10909489	12	0	9	0	10
17	10978211	22	0	9	0	10
18	11023631	9	0	9	0	10
19	11030265	5	0	9	0	10
20						

Fig. 15: Example of data separation.

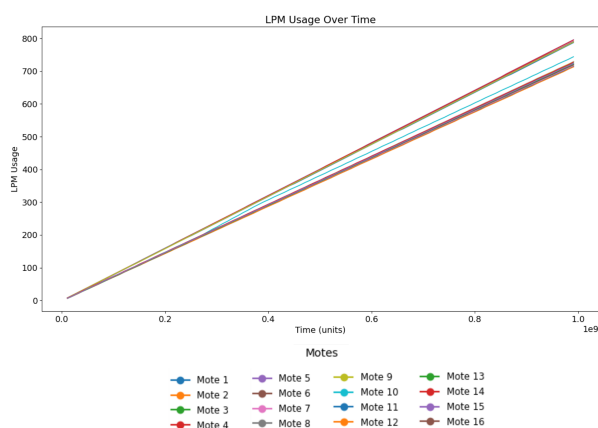
V. DATA SEPARATION AND PLOTTING

In the next section, we describe how network performance data is visualized. First, we extract the relevant metrics from our log files using a script called log-to-csv.py. This script parses raw log data and filters out the needed information. The data preprocessing step allows only the pertinent data to be captured for analysis, setting the scene for meaningful insights (Fig. 15).

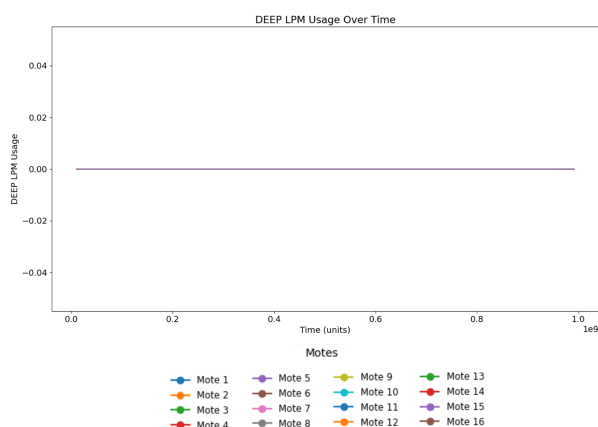
After the data is divided, we use the Pandas library to assist with plotting. Pandas provides an efficient environment for manipulating and analyzing our data, allowing us to create several



(a) CPU time.



(b) LPM time



(c) Deep LPM time.

Fig. 16: Indicative plots.

types of visualizations. Using Pandas' data-handling abilities, we can plot data in a clear and informative manner, providing further insight into the network's performance and behavior. We plotted the data using the script `csv-to-png.py`. Example results of a simulation run are given in Fig. 16. The plots shown in Fig. 16 reflect the results of our experiments. We kept the network topology, the number of motes, the transmission, and the interference range fixed, while we varied the frequency of sending messages, which impacted the parameters of CPU, LPM, and Deep LPM.

VI. CONCLUSIONS

Our research investigates the challenge of automating experiments in Cooja, which is otherwise highly manual. We propose an approach that improves efficiency by automating most aspects of Cooja's experiments. It allows for the systematic variation and analysis to be performed of meaningful parameters in an efficient manner that is not prone to human errors. In particular, our approach automates the configuration of the number of motes, adjustment of the transmission and interference ranges, variation of message sending frequency, implementation of various network topologies, and logging and extraction of metrics such as CPU usage, LPM, Deep LPM, Radio Listen, and Radio Transmit data. This comprehensive automation enables far more efficient experimentation and data gathering. In the future, we plan to extend automation to include more varied parameters and newer data scenarios that represent complex use cases. We also plan to refine the approach to automation in order to make it more scalable and further improve efficiency when conducting large-scale experiments.

ACKNOWLEDGMENTS

The work of G. Grigoryan and H. Khachatrian was supported by the Higher Education and Science Committee of RA (22rl-052 DISTAL). The work of T. P. Raptis was partly supported by the bilateral agreement CNR/IVI OLINA 2024-2025.

REFERENCES

- [1] G. D'Angelo, S. Ferretti, and V. Ghini, "Simulation of the internet of things," in *2016 International Conference on High Performance Computing & Simulation (HPCS)*, 2016, pp. 1–8.
- [2] T. P. Raptis, A. Passarella, and M. Conti, "Distributed data access in industrial edge networks," *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 5, pp. 915–927, 2020.
- [3] F. Osterlind, A. Dunkels, J. Eriksson, N. Finne, and T. Voigt, "Cross-level sensor network simulation with cooja," in *Proceedings. 2006 31st IEEE Conference on Local Computer Networks*, 2006, pp. 641–648.
- [4] V. Ajayy and V. Ranga, "Performance analysis of RPL protocol in different nodes positioning using Contiki Cooja," *International Journal of Information Technology*, Jun. 2024. [Online]. Available: <https://doi.org/10.1007/s41870-024-01985-w>
- [5] H. Khachatrian, R. Mkrtchyan, and T. P. Raptis, "Outdoor environment reconstruction with deep learning on radio propagation paths," in *2024 International Wireless Communications and Mobile Computing (IWCMC)*, 2024, pp. 1498–1503.

- [6] R. Darbinyan, H. Khachatrian, R. Mkrtchyan, and T. P. Raptis, "MI-based approaches for wireless nlos localization: Input representations and uncertainty estimation," in *2023 IEEE Ninth International Conference on Big Data Computing Service and Applications (BigDataService)*, 2023, pp. 87–94.
- [7] I. H. Turan, D. Yildiz, and S. Demirci, "Protocol stack-aware comparison of centroid localization algorithms based on anchor density using cooja," in *2023 International Conference on Smart Applications, Communications and Networking (SmartNets)*, 2023, pp. 1–6.
- [8] L. F. Perrone, C. J. Kenna, and B. C. Ward, "Enhancing the credibility of wireless network simulations with experiment automation," in *2008 IEEE International Conference on Wireless and Mobile Computing, Networking and Communications*, 2008, pp. 631–637.
- [9] N. Finne, J. Eriksson, T. Voigt, G. Suciu, M.-A. Sachian, J. Ko, and H. Keipour, "Multi-trace: Multi-level data trace generation with the cooja simulator," in *2021 17th International Conference on Distributed Computing in Sensor Systems (DCOSS)*, 2021, pp. 390–395.
- [10] H. Keipour, S. Hazra, N. Finne, and T. Voigt, "Generalizing supervised learning for intrusion detection in iot mesh networks," in *Ubiquitous Security*, G. Wang, K.-K. R. Choo, R. K. L. Ko, Y. Xu, and B. Crispo, Eds. Singapore: Springer Singapore, 2022, pp. 214–228.
- [11] I. Essop, J. C. Ribeiro, M. Papaioannou, G. Zachos, G. Mantas, and J. Rodriguez, "Generating datasets for anomaly-based intrusion detection systems in iot and industrial iot networks," *Sensors*, vol. 21, no. 4, 2021. [Online]. Available: <https://www.mdpi.com/1424-8220/21/4/1528>
- [12] A. Bumb, B. Iancu, and E. Cebuc, "Extending cooja simulator with real weather and soil data," in *2018 17th RoEduNet Conference: Networking in Education and Research (RoEduNet)*, 2018, pp. 1–5.